

Project Configuration

Storefront Next uses a centralized, type-safe configuration system that gives you:

- IDE autocomplete: Full TypeScript support with suggestions as you type.
- Environment overrides: Customize settings per environment without code changes.
- Validation: Catch configuration errors at startup, not at run time.
- Security by design: Clear separation between public and server-only values.

All configuration lives in a single file (`config.server.ts`) with defaults that you can override by using environment variables.

Quick Start

Storefront Next provides two APIs for accessing configuration.

```
1 // In loaders and actions - pass context
2 import { getConfig } from "@config";
3
4 export function loader({ context }: LoaderContext) {
5   const config = getConfig(context);
6   return { productsPerPage: config.global.productsPerPage };
7 }
8
9 // In React components - use the hook
10 import { useConfig } from "@config";
11
12 function ProductGrid() {
13   const config = useConfig();
14   return <div>Showing {config.global.productsPerPage} products</div>;
15 }
```

Configuration File

The main configuration file is `config.server.ts` in your project root.

```
1 // config.server.ts
2 import { defineConfig } from "@config/schemas";
3
```

Contents

Quick Start

- [Configuration File](#)
- [Minimum Required Configuration](#)
- [Environment Variable Overrides](#)
- [Security: Public vs. Private Configuration](#)
- [Accessing Configuration](#)
- [Adding New Configuration](#)
- [Deployment](#)
- [Troubleshooting](#)



```
9 runtime: {
10   ssrOnly: [],
11   ssrParameters: { ssrFunctionNodeVersion
12 },
13 app: {
14   commerce: {
15     /* Commerce API settings */
16   },
17   site: {
18     /* Locale, currency, features */
19   },
20   global: {
21     /* Branding, product listing defaults
22   },
23   pages: {
24     /* Page-specific settings */
25   },
26   performance: {
27     /* Image optimization, caching */
28   },
29   engagement: {
30     /* Analytics adapters */
31   },
32 },
33 };
```

For descriptions of the configuration options, see [Configuration Options Reference](#) in the `storefront-next-template` GitHub repo.

The configuration is organized into three main sections.

Section	Purpose	Client Access
metadata	Project name and slug for deployment	Server only
runtime	SSR and deployment settings	Server only
app	Application settings (commerce, features, UI)	Server and client

The `app` section is automatically made available to client-side code. The `metadata` and `runtime` sections remain server-only.

The `runtime.ssrOnly` option accepts an array of glob patterns for files that must be available on the server but aren't publicly accessible from the client. Use this option for server-side resources that aren't exposed via public URLs.

Warning

Don't import `config.server.ts` directly in your app code. While the `.server.ts` suffix prevents the file from being bundled



available in the correct context.

Minimum Required Configuration

To start the storefront, set B2C Commerce API credentials in a `.env` file at your project root. All the other settings have a working default in `config.server.ts`. Declare them only if you want to override their values.

The Storefront Next template includes an `.env.default` file pre-filled with public demo credentials. Copy it to `.env` to get started.



```
1 cp .env.default .env
```

Public Client (Default)

A public SLAS client needs three variables. These values are safe to expose to the browser, so they use the `PUBLIC__` prefix.



```
1 PUBLIC__app__commerce__api__clientId=your-c  
2 PUBLIC__app__commerce__api__organizationId=  
3 PUBLIC__app__commerce__api__shortCode=your-
```

This table describes each variable.

Variable
<code>PUBLIC__app__commerce__api__clientId</code>
<code>PUBLIC__app__commerce__api__organizationId</code>
<code>PUBLIC__app__commerce__api__shortCode</code>

Private Client

[Try for free](#)

```
1 # Client-safe values (PUBLIC__ prefix bundled in storefront)
2 PUBLIC__app_commerce_api_clientId=your-client-id
3 PUBLIC__app_commerce_api_organizationId=your-organization-id
4 PUBLIC__app_commerce_api_shortcode=your-shortcode
5 PUBLIC__app_commerce_api_privateKeyEnabled=true
6
7 # Server-only secret – never use the PUBLIC__ prefix
8 COMMERCE_API_SLAS_SECRET=your-slas-secret
```



Warning

Don't add `COMMERCE_API_SLAS_SECRET` to `config.server.ts` and don't give it a `PUBLIC__` prefix. Read it directly from `process.env` in server-side code, such as loaders, actions, and middleware. Anything with the `PUBLIC__` prefix is bundled into the browser-visible JavaScript.

For SLAS private client setup in B2C Commerce, see [Authorization for Shopper APIs](#).

To deploy your storefront to Managed Runtime, also set the deployment variables that the Storefront Next CLI reads. See [Storefront Next CLI](#).

Optional Configuration

Every other setting has a working default in `config.server.ts`. To override one, use a `PUBLIC__` environment variable with the path syntax described in [Environment Variable Overrides](#).

For feature-specific configuration, see the dedicated guides.

- [Configure Multisite URLs](#) – sites, locales, currencies, and URL patterns
- [Set Up Hybrid Proxy Locally](#) – `HYBRID_PROXY_*`, `SFCC_ORIGIN`, routing rules
- [Passwordless Login for Storefront Next](#) – passwordless login, OTP request, and reset password modes
- [Shopper Context](#) – qualifier-based personalization
- [Shopping Agent for Storefront Next](#) – embedded chat configuration

Environment Variable Overrides

[Try for free](#)

Path Syntax

Use double underscores (`__`) to navigate nested configuration paths.

```
1 # Environment variable      → Config path
2 PUBLIC__app__site__locale   → config.app__site__locale
3 PUBLIC__app__commerce__api__clientId → config.app__commerce__api__clientId
```

Automatic Type Parsing

Values are automatically parsed to match the expected type.

```
1 PUBLIC__app__global__productListing__product
2 PUBLIC__app__site__features__guestCheckout=
3 PUBLIC__app__site__features__socialLogin=
```

You can also set entire nested objects using JSON.

```
1 PUBLIC__app__site__features__socialLogin={}
```

Case Insensitivity

Path matching is case-insensitive, so all these variables work.

```
1 PUBLIC__app__site__locale=en-US
2 PUBLIC__APP__SITE__LOCALE=en-US
3 PUBLIC__App__Site__Locale=en-US
```

Merge Behavior

Environment variables are deep merged into the defaults from `config.server.ts`. Only the values you specify are overridden. Everything else keeps its default.

```
1 // Default in config.server.ts
2 pages: {
3   cart: {
4     quantityUpdateDebounce: 750,
5     maxQuantityPerItem: 999,
6     enableRemoveConfirmation: true,
7   }
8 }
```

[Try for free](#)

```
13   cart: {
14     quantityUpdateDebounce: 1000, // c
15     maxQuantityPerItem: 999,     //
16     enableRemoveConfirmation: true, //
17   }
18 }
```

Security: Public vs. Private Configuration

The `PUBLIC__` prefix indicates values that are **safe to expose to the browser**. These values are bundled into client-side JavaScript.



Important

Don't use `PUBLIC__` for secrets, API keys, passwords, or authentication tokens. These values are visible to anyone who uses your site.

Use <code>PUBLIC__</code> For	Use Non-Prefixed For
Client IDs	API secrets
Site IDs	Private keys
Locales and currencies	Database credentials
Feature flags	Authentication tokens
Public API endpoints	SLAS secrets

For server-only secrets, use environment variables without the `PUBLIC__` prefix and read them directly from `process.env`.



```
1 # .env - Server-only secret (no PUBLIC__ pr
2 COMMERCE_API_SLAS_SECRET=your-secret-here
```



```
1 // In server-side code only
2 const slasSecret = process.env.COMMERCE_API
```

[Try for free](#)

In Loaders and Actions

Use `getConfig()` with the router context.

```
1 import { getConfig } from "@config";
2
3 export function loader({ context }: LoaderFunctionArgs) {
4   const config = getConfig(context);
5   const { clientId, siteId } = config.commerce;
6   return { siteId };
7 }
8
9 export async function action({ context, request }: ActionFunctionArgs) {
10   const config = getConfig(context);
11   // Use config values...
12 }
```

In React Components

Use the `useConfig()` hook:

```
1 import { useConfig } from "@config";
2
3 function Header() {
4   const config = useConfig();
5   return <h1>{config.global.branding.name}</h1>;
6 }
```

In Client Loaders

Client loaders don't have access to router context.

Call `getConfig()` without arguments.

```
1 export function clientLoader() {
2   const config = getConfig(); // Uses window.__CONFIG__
3   return { locale: config.site.locale };
4 }
```

Adding New Configuration

To add a new configuration value, follow these steps.

1. Define the Type

Add the type definition in `src/config/schema.ts`:

```
1 import { z } from "zod";
2
3 export const schema = z.object({
4   // ...
5 });
```



```
5 // ...
6     maxItems: number;
7   };
8 };
9 };
```

2. Set the Default Value

Add the default in `config.server.ts`:

```
1 export default defineConfig({
2   app: {
3     myFeature: {
4       enabled: false,
5       maxItems: 10,
6     },
7   },
8 });
```

3. Override via Environment Variables

No code changes needed. Use the `PUBLIC__` prefix.

```
1 PUBLIC__app__myFeature__enabled=true
2 PUBLIC__app__myFeature__maxItems=25
```

4. Use in Your Code

```
1 // In a component
2 const config = useConfig();
3 if (config.myFeature.enabled) {
4   // Feature is enabled
5 }
6
7 // In a loader
8 const config = getConfig(context);
9 const limit = config.myFeature.maxItems;
```

Deployment

When deploying to Managed Runtime (MRT), set your environment variables in the Runtime Admin.

1. Log in to the Runtime Admin.
2. Navigate to your project → Environment Variables.
3. Add the required `PUBLIC__` variables.

**Note**

MRT has limits: variable names max 512 characters, total `PUBLIC__` values max 32 KB. Use JSON to consolidate related settings if needed.

Troubleshooting

Changed `.env` but Nothing Happened?

Restart your dev server. Environment variables are loaded at startup.

Environment Variable Not Working?

- Verify the variable name starts with `PUBLIC__` (double underscore after PUBLIC)
- Check the `.env` file is in the project root
- Ensure that the path exists in `config.server.ts`—you can only override existing paths

Type Errors After Adding Configuration?

Update both `src/config/schema.ts` (types) and `config.server.ts` (defaults) to match.

App Won't Start—Missing Credentials?

Copy `.env.default` to `.env` and set the required B2C Commerce credentials. See [Minimum Required Configuration](#).

Path Validation Error?

The configuration system validates that environment variable paths exist in your config. If you see an error like `"local" doesn't exist`, check for typos. The system suggests similar valid paths when possible.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

[Try for free](#)

MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

Component Library
APIs
Trailhead
Sample Apps
AppExchange

Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

